Observer Design Pattern

Intent

Define a **one-to-many** dependency between objects so that when one object (the **subject / observable**) changes state, all its dependents (the **observers**) are notified — without the subject knowing the concrete type of any observer. It's the "publish/subscribe" relationship: subscribers register with a publisher, and the publisher broadcasts changes to whoever is currently listening.

Here the subject is a `MessagePublisher` that holds a message, and the observers are **subscribers** that want to react whenever the message changes.

UML class diagram

```
 <<interface>> Observable            <<interface>> Observer
 | +attach(Observer)     |           | +update() |
 | +detach(Observer)     |              ^      ^
 | +notifyUpdate()       |              |      |
 +-----------^-----------+     MessageSubscriberOne  MessageSubscriberTwo
             |                  (attach(this) in ctor; pulls getMsg())
     MessagePublisher
     | -observers : List<Observer>
     | -msg : String
     | +setMsg / +getMsg
     +--> notifyUpdate() -> for each observer.update()
```

The players

```
observer/Observer                          the observer contract: update()
observer/concretObserver/MessageSubscriberOne   concrete observers — print the message on update()
                        /MessageSubscriberTwo

subject/Observable                         the subject contract: attach / detach / notifyUpdate
subject/concretSubject/MessagePublisher    concrete subject — keeps a list of observers + the message

DesignPatternsApplication                  the demo — wires a publisher and two subscribers
```

Two contracts facing each other:

- `Observable` = the publisher side (register/unregister/broadcast).
- `Observer` = the subscriber side (be notified).

The code, line by line

`Observer` — the subscriber contract

```
public interface Observer {
        public void update();
}
```

- One method: `update()` — "something you care about changed; react now." Every subscriber implements this.
- The subject will call `update()` on each registered observer. Because the subject only knows this interface, it can notify *any* observer without knowing its class — that's the decoupling.

`Observable` — the publisher contract

```
public interface Observable {
        public void attach(Observer observer);
        public void detach(Observer observer);
        public void notifyUpdate();
}
```

- `attach(observer)` — subscribe: add an observer to the notify list.
- `detach(observer)` — unsubscribe: remove one.
- `notifyUpdate()` — broadcast: tell every currently-registered observer to `update()`.

These three methods are the entire publisher side of the pattern: manage a dynamic subscriber list and fan out notifications to it.

`MessagePublisher` — the concrete subject

```
public class MessagePublisher implements Observable {

        private List<Observer> observers = new ArrayList<>();
        private String msg;

        @Override public void attach(Observer observer) { observers.add(observer); }
        @Override public void detach(Observer observer) { observers.remove(observer); }

        @Override public void notifyUpdate() {
                observers.stream().forEach(observer -> observer.update());
        }

        public String getMsg()            { return msg; }
        public void   setMsg(String msg)  { this.msg = msg; }
}
```

- **`private List<Observer> observers`** — the subscriber registry. It holds observers **by the interface type**, so the publisher never knows (or cares) that they're `MessageSubscriberOne` / `Two`. Observers can join and leave at runtime — the list is dynamic.
- **`attach` / `detach`** — add/remove from that list.
- **`notifyUpdate()`** — iterates the list and calls `update()` on each observer. This is the "broadcast": one call fans out to all subscribers. (It uses a stream `forEach`, equivalent to a plain loop over the list.)
- **`msg` + `getMsg()` / `setMsg()`** — the piece of state the observers care about. `getMsg()` is how an observer *pulls* the current value when notified (this is the **pull model**; see *Why* below).

The concrete observers

```
public class MessageSubscriberOne implements Observer {

        private MessagePublisher observable;

        public MessageSubscriberOne(MessagePublisher _obserObservable) {
                this.observable = _obserObservable;
                this.observable.attach(this);      // self-register with the publisher
        }

        @Override
        public void update() {
                System.out.println(this.observable.getMsg());
        }
}
```

- **The constructor takes the publisher and immediately calls `observable.attach(this)`** — so a subscriber **registers itself** at construction time. As soon as you `new` a `MessageSubscriberOne(publisher)`, it's on the publisher's notify list.
- **It keeps a reference to the publisher** so that, when notified, it can **pull** the current message via `observable.getMsg()`.
- **`update()`** is the reaction: print the publisher's current message. `MessageSubscriberTwo` is identical (its `update()` also prints `getMsg()`); having two shows that one change can drive many independent reactions.

`DesignPatternsApplication` — the demo

```
MessagePublisher publisher = new MessagePublisher();
publisher.setMsg("---I am going to change-----");

MessageSubscriberOne messageSubscriberOne = new MessageSubscriberOne(publisher);
messageSubscriberOne.update();

publisher.setMsg("-----Am i going to change------");
MessageSubscriberTwo messageSubscriberTwo = new MessageSubscriberTwo(publisher);
messageSubscriberTwo.update();
```

- Create the publisher and set a message.
- Create `MessageSubscriberOne` — its constructor **attaches** it to the publisher. Then `update()` is called and it prints the current message.
- Change the message, create `MessageSubscriberTwo` (which attaches itself), and call its `update()`.

Console output:

```
---I am going to change-----
-----Am i going to change------
```

Important: what this demo does *not* do (honest read of the code)

The classic Observer flow is: **change state → the subject calls `notifyUpdate()` → every observer's `update()` fires automatically.** In this demo that automatic broadcast is **never triggered**:

- `setMsg(...)` only stores the string. It does **not** call `notifyUpdate()`, so changing the message does not notify anyone on its own.
- `publisher.notifyUpdate()` is **never called** anywhere. Instead the demo calls each observer's `update()` **manually** (`messageSubscriberOne.update()`).

So the subscription machinery (`attach`, the observer list, `notifyUpdate`) is fully built, but the demo exercises it by hand rather than letting the subject drive it. A consequence: after the second `setMsg(...)`, `MessageSubscriberOne` is **not** re-notified — nothing calls it again — which is why it doesn't print the new message.

How it's *meant* to work (the one-line change that makes it a true Observer, described here, not applied to the code): have `setMsg` end with `notifyUpdate()`:

```
public void setMsg(String msg) {
        this.msg = msg;
        notifyUpdate();     // <-- broadcast to all attached observers automatically
}
```

Then a single `publisher.setMsg("…")` would cause **every attached subscriber** to print the new value, with no manual `update()` calls — which is the whole point of the pattern. The README documents the current behavior faithfully; the code is left unchanged as requested.

Why the design decisions

Why Observer instead of the publisher calling subscribers directly?

Without it, the publisher would need a hard-coded reference to every subscriber and would call each one by name — tightly coupling it to specific classes, and forcing a code change every time a new subscriber appears. Observer inverts this: subscribers **register themselves**, and the publisher just walks an anonymous list. New subscriber types can be added with **zero changes** to the publisher.

Why does the subject hold `List<Observer>` (the interface), not concrete subscribers?

So the subject is decoupled from who's listening. It can notify anything that implements `Observer`, present or future. This is "program to an interface" again — the publisher's broadcast code never mentions a concrete subscriber class.

Why do observers `attach(this)` in their constructor?

It makes subscription automatic and atomic: the moment an observer exists, it's already listening — you can't forget to register it. (Trade-off: it also couples the observer to a *specific* publisher passed into its constructor, and leaking `this` from a constructor is something to be careful with in multithreaded code. A common alternative is to attach explicitly from the outside: `publisher.attach(subscriber)`.)

Why the "pull" model (`getMsg()`) instead of pushing the data into `update()`?

There are two flavors of Observer:

- **Push** — the subject passes the changed data as an argument: `update(String newMsg)`.
- **Pull (this module)** — the subject just says "something changed" via `update()`, and the observer **pulls** whatever it needs from the subject (`observable.getMsg()`).

Pull keeps the `Observer` interface tiny and generic (one no-arg method works for any kind of change), and lets each observer decide exactly which parts of the subject's state it cares about. The cost is that every observer must hold a reference back to the subject to pull from it — which is exactly why these subscribers store `observable`.

Why `detach` even though the demo never calls it?

Lifecycle management. Real observers come and go; `detach` lets a subscriber stop listening so the subject doesn't keep notifying (and keep alive) observers that no longer care. Omitting detach in long-running systems is a classic source of the **lapsed-listener memory leak**.

Execution flow (the demo)

```
main
 │
 ├── new MessagePublisher()                      observers = []          msg = null
 ├── publisher.setMsg("---I am going to change-----")   msg set (no notify)
 │
 ├── new MessageSubscriberOne(publisher)
 │       └── publisher.attach(this)              observers = [One]
 ├── messageSubscriberOne.update()
 │       └── prints observable.getMsg()  → "---I am going to change-----"
 │
 ├── publisher.setMsg("-----Am i going to change------")  msg changed (no notify → One NOT re-called)
 │
 ├── new MessageSubscriberTwo(publisher)
 │       └── publisher.attach(this)              observers = [One, Two]
 └── messageSubscriberTwo.update()
         └── prints observable.getMsg()  → "-----Am i going to change------"
```

Note how `notifyUpdate()` (the broadcast to the whole `observers` list) is never reached — each `update()` is a manual, single-observer call. With the `setMsg → notifyUpdate` wiring described above, the second `setMsg` alone would have printed the new message from **both** subscribers.

Notes / possible extensions (not changed in the code)

- **Two `update()` methods carry leftover `// TODO Auto-generated method stub` comments** from the IDE (in `MessageSubscriberTwo.update` and inside `MessagePublisher`). They're harmless placeholders and don't affect behavior.
- **Thread safety.** A plain `ArrayList` iterated during `notifyUpdate()` isn't safe if observers attach/detach concurrently (or during notification). Production code often uses a `CopyOnWriteArrayList`.
- **`SpringApplication.run(...)`** boots a context but is unrelated to the pattern; the wiring here is all manual. (Spring's own `ApplicationEvent` / `@EventListener` is essentially the Observer pattern provided by the framework.)

Relationship to the other Behavioral patterns

- **Observer (this module)** — one subject broadcasts to **many** observers that registered themselves; a state change fans out to all listeners.
- **Chain of Responsibility** — a request travels to **one** handler along a chain, not broadcast to all.
- **Mediator** — centralizes many-to-many communication in a hub, instead of subjects notifying observers directly.
- **State / Strategy** — a context delegates to a *single* swapped-in object, not a list of listeners.

Reach for Observer when a change in one object must be reflected in an **open-ended set** of others, and you want the source to stay ignorant of who's listening — event systems, UI data-binding, and pub/sub messaging are the canonical uses.